

hw2

Yi-Ting Tsai

This is the link to my repo: https://github.com/ytting920410/STATS_506

Import library

```
library(microbenchmark)
```

Warning: package 'microbenchmark' was built under R version 4.5.3

```
library(roxygen2)
```

Warning: package 'roxygen2' was built under R version 4.5.3

Problem 1: Random Walk with Momentum

Create a function to check the input of every steps

```
#' Sanitize inputs for a momentum random walk
#'  
#'  
#' Checks whether the inputs for a momentum random walk satisfy the required  
#' conditions.  
#'  
#'  
#' @param n A positive integer specifying the number of steps.  
#' @param p A numeric value between 0 and 1.  
#'  
#'  
#' @return This function returns \code{NULL} invisibly if both inputs are  
#' valid. Otherwise, it stops with an error message.  
#'
```

```

#' @keywords internal
check_input <- function(n, p){
  if(length(n) != 1 || !is.numeric(n) || is.na(n) ||
      n < 1 || !is.finite(n) || n != floor(n)){
    stop('n must be a positive integer') # check n
  }
  if(length(p) != 1 || !is.numeric(p) || is.na(n) ||
      p < 0 || p > 1 || ! is.finite(p)){
    stop('p must be a real number between 0 and 1') # check p
  }
}

```

a. Simulation function

```

#' Simulate a momentum random walk
#'
#' Simulates an \code{n}-step two-dimensional random walk. The first
#' direction is chosen randomly. Each later step keeps the previous direction
#' with probability \code{p}; otherwise, a new direction is chosen randomly.
#'
#' @param n A positive integer indicating the number of steps.
#' @param p A number between 0 and 1 indicating the momentum probability.
#'
#' @return A named vector containing the final x and y coordinates.
#'
#' @examples
#' set.seed(123)
#' momentum_walk(10, 0.6)
momentum_walk <- function(n, p){
  check_input(n, p) # sanitize the input
  steps <- matrix( # set possible moves
    c(1, 0,
      0, 1,
      -1, 0,
      0, -1),
    ncol = 2,
    byrow = TRUE
  )

  direction <- sample.int(4, size = 1) # the first step

```

```

x <- steps[direction, 1] # the first step on the x axis
y <- steps[direction, 2] # the first step on the y axis
n <- n - 1 # we've done one step, so we have n - 1 step left
while(n > 0){ # n here is the number of remaining step we have
  if(runif(1) > p){ # for 1 - p chances we have to choose direction again
    direction <- sample.int(4, size = 1)
  }
  x <- x + steps[direction, 1] # update our position on x axis
  y <- y + steps[direction, 2] # update our position on y axis
  n <- n - 1 # n - 1 steps remain
}
c(x = x, y = y)
}

```

Test the momentum_walk function

```

set.seed(44)
momentum_walk(10, 0.6)

```

```

  x  y
10  0

```

```

momentum_walk(100, 0.4)

```

```

  x  y
-20  8

```

b. implement the simulation

Version 1

Same as momentum_walk function above

```

#' Simulate a momentum random walk
#'
#' Simulates an n-step two-dimensional random walk with loop. The first
#' direction is chosen randomly. Each later step keeps the previous direction
#' with probability p; otherwise, a new direction is chosen randomly.
#'
#' @param n A positive integer indicating the number of steps.
#' @param p A number between 0 and 1 indicating the momentum probability.
#'
#' @return A named vector containing the final x and y coordinates.
#'
#' @examples
#' set.seed(123)
#' momentum_walk(10, 0.6)
momentum_walk1 <- function(n, p){
  check_input(n, p) # sanitize the input
  steps <- matrix( # set possible moves
    c(1, 0,
      0, 1,
      -1, 0,
      0, -1),
    ncol = 2,
    byrow = TRUE
  )

  direction <- sample.int(4, size = 1) # the first step
  x <- steps[direction, 1] # the first step on the x axis
  y <- steps[direction, 2] # the first step on the y axis
  n <- n - 1 # we've done one step, so we have n - 1 step left
  while(n > 0){ # n here is the number of remaining step we have
    if(runif(1) > p){ # for 1 - p chances we have to choose direction again
      direction <- sample.int(4, size = 1)
    }
    x <- x + steps[direction, 1] # update our position on x axis
    y <- y + steps[direction, 2] # update our position on y axis
    n <- n - 1 # n - 1 steps remain
  }
  c(x = x, y = y)
}

```

Version 2

```
#' Simulate a momentum random walk
#
#' Simulates an \code{n}-step two-dimensional random walk with lapply. The first
#' direction is chosen randomly. Each later step keeps the previous direction
#' with probability \code{p}; otherwise, a new direction is chosen randomly.
#
#' @param n A positive integer indicating the number of steps.
#' @param p A number between 0 and 1 indicating the momentum probability.
#
#' @return A named vector containing the final x and y coordinates.
#
#' @examples
#' set.seed(123)
#' momentum_walk(10, 0.6)
momentum_walk2 <- function(n, p){
  check_input(n, p) # sanitize the input
  steps <- matrix( # set possible moves
    c(1, 0,
      0, 1,
      -1, 0,
      0, -1),
    ncol = 2,
    byrow = TRUE
  )

  direction <- sample.int(4, size = 1) # the first step
  all_direction <- c( # produce all n steps we will go
    direction, # add the first step
    if(n >= 2){ # if we have n = 1, we don't need further action
      lapply(2: n, function(i){ # for n >= 2, we have to calculate steps
        if(runif(1) > p){ # for 1 - p chances we have to choose direction again
          direction <- sample.int(4, size = 1)
        }
        return(direction) # get next step
      })
    }else{
      integer(0) # no action
    }
  )
}
```

```

final_pos <- colSums(steps[unlist(all_direction), , drop = FALSE])
# calculate our final position by adding all the moves we made
c(x = final_pos[1], y = final_pos[2])
}

```

Test the momentum_walk2 function

```

set.seed(44)
momentum_walk2(1, 0.6)

```

```

x y
1 0

```

```

momentum_walk2(100, 0.4)

```

```

  x    y
-17    7

```

c. Reproducibility

```

set.seed(128)
result1_f1 <- momentum_walk1(10, 0.6)
set.seed(128)
result2_f1 <- momentum_walk1(10, 0.6)
set.seed(128)
result1_f2 <- momentum_walk1(1000, 0.6)
set.seed(128)
result2_f2 <- momentum_walk1(1000, 0.6)

result1_f1 == result2_f1 # test if these results are the same

```

```

  x    y
TRUE TRUE

```

```
result1_f2 == result2_f2 # test if these results are the same
```

```
      x      y  
TRUE TRUE
```

set.seed sets the random number generator to a fixed initial state. As long as the seed, functions used are called in the same order, the same result will be reproduced.

d. Does momentum mater?

```
set.seed(32)  
resultsD <- sapply(c(0, 0.5, 0.9), function(p) momentum_walk1(1000, p))  
# run momentum walk 1000 times for p = 0, 0.5, 0.9  
distance <- apply(t(resultsD), 1, function(pos) sqrt(sum(pos ^ 2)))  
# calculate each distances from (0, 0) to (x, y)  
resultsDdf <- data.frame(  
  p = c(0, 0.5, 0.9),  
  x = resultsD['x', ],  
  y = resultsD['y', ],  
  distance = distance  
)  
resultsDdf
```

```
      p      x      y distance  
1 0.0 -26 -10 27.85678  
2 0.5  30  28 41.03657  
3 0.9  53  45 69.52697
```

Increasing p tends to increase the final distance. Having higher p, the simulation tends to move toward the same way.

e. Monte Carlo simulation

```
p <- c(0, 0.25, 0.5, 0.75, 0.9) # for each of p  
mean_distance <- sapply(  
  p,  
  function(prob){ # do momentum walk 1000 times
```

```

distance <- replicate(
  1000,
  {
    position <- momentum_walk1(1000, prob) # do momentum walk with 1000 steps
    sqrt(sum(position ^ 2)) # calculate distance
  }
)
mean_distance <- mean(distance) # get the mean of 1000 tries
}
)
result_md <- data.frame(
  p = p,
  distance = mean_distance
)
result_md

```

	p	distance
1	0.00	27.93875
2	0.25	36.28528
3	0.50	48.78594
4	0.75	74.00012
5	0.90	123.38382

The mean of final distance increases as p increases.

f. Compare computational performance

```

BM_1000 <- microbenchmark( # test both methods with 1000 times
  loop = momentum_walk1(1000, 0.6),
  reduce = momentum_walk2(1000, 0.6),
  time = 100L
)

```

Warning in microbenchmark(loop = momentum_walk1(1000, 0.6), reduce = momentum_walk2(1000, : Could not measure a positive execution time for 17 evaluations.

```
BM_100000 <- microbenchmark( # test both methods with 1000 times
  loop = momentum_walk1(100000, 0.6),
  reduce = momentum_walk2(100000, 0.6),
  time = 20L
)
```

Warning in microbenchmark(loop = momentum_walk1(1e+05, 0.6), reduce = momentum_walk2(1e+05, : Could not measure a positive execution time for 15 evaluations.

```
print(BM_1000)
```

Unit: nanoseconds

expr	min	lq	mean	median	uq	max	neval
loop	2499202	2978300	4039171.94	3474551.0	4623352	8254400	100
reduce	3525801	4272901	5475046.96	4840150.5	6296902	14139801	100
time	0	1	167.84	101.5	201	1601	100

```
print(BM_100000)
```

Unit: nanoseconds

expr	min	lq	mean	median	uq	max	neval
loop	372364900	420002551	443840917.01	443028301	463754002	584252101	100
reduce	537906201	579685151	613828731.06	618166901	642913700	731710900	100
time	0	1	129.86	101	201	601	100

- i. It seems version 1 is faster.
- ii. The comparison result is the same as the smaller input.
- iii. Version 2 has additional function calls and version 1 just updates the direction and position directly. So version 1 run faster.

Problem 2: Monte Carlo Simulation of Campus Bus Arrivals

```
set.seed(256)

n <- 10000 # 10000 days

time68 <- matrix( # simulated number of passengers in each hours from 6AM to 8AM
```

```

    rpois(n * 3, lambda = 4),
    nrow = n,
    ncol = 3
  )

time9 <- matrix( # simulated number of passengers in 9AM
  rnorm(n, mean = 45, sd = 4),
  nrow = n,
  ncol = 1
)

time1015 <- matrix( # simulated number of passengers in each hours from 10AM to 3PM
  rpois(n * 6, lambda = 15),
  nrow = n,
  ncol = 6
)

time16 <- matrix( # simulated number of passengers in 4PM
  rnorm(n, mean = 55, sd = 5),
  nrow = n,
  ncol = 1
)

time1722 <- matrix( # simulated number of passengers in each hours from 5PM to 10PM
  rpois(n * 6, lambda = 9),
  nrow = n,
  ncol = 1
)

```

Warning in matrix(rpois(n * 6, lambda = 9), nrow = n, ncol = 1): data length differs from size of matrix: [60000 != 10000 x 1]

```

total <- rowSums(time68) + time9 + rowSums(time1015) + time16 + rowSums(time1722)
# sum of each day
mean(total) # calculate mean of 10000 days of data

```

```
[1] 211.0037
```

The simulated estimation is about 211 passengers per day.

Problem 3: Linear Regression

a. Data preparation and de-identification

```
youtube <- read.csv( # read data
  "https://raw.githubusercontent.com/rfordatascience/tidytuesday/master/data/2021/2021-03-02,
)

youtube_deiden <- youtube[!(names(youtube) %in% c(
  'brand',
  'title',
  'link',
  'youtube_link',
  'channel_title',
  'category_id',
  'published_at',
  'thumbnail',
  'description'
))] # remove the columns we do not need
dim(youtube_deiden)
```

```
[1] 247  18
```

```
names(youtube_deiden)
```

```
[1] "year" "superbowl_ads_dot_com_url"
[3] "youtube_url" "funny"
[5] "show_product_quickly" "patriotic"
[7] "celebrity" "danger"
[9] "animals" "use_sex"
[11] "id" "kind"
[13] "etag" "view_count"
[15] "like_count" "dislike_count"
[17] "favorite_count" "comment_count"
```

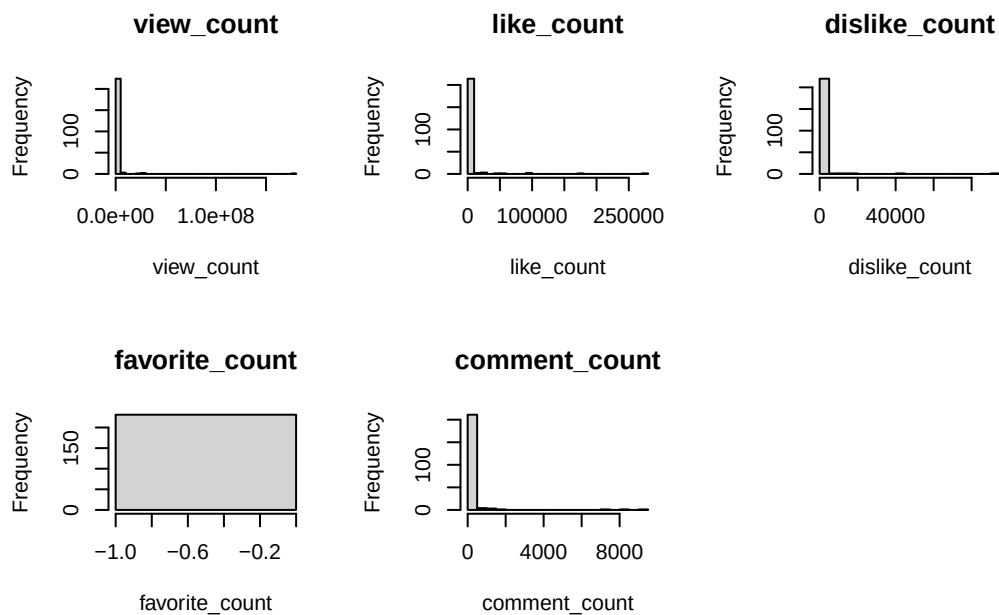
There are 247 rows and 18 columns

b. Exploring potential outcomes

```
outcome <- c(
  'view_count',
  'like_count',
  'dislike_count',
  'favorite_count',
  'comment_count'
)

par(mfrow = c(2, 3)) # draw the distribution and print the count of favorite_count
for(i in outcome){
  hist(youtube_deiden[[i]],
      main = i,
      xlab = i,
      breaks = 30
  )
}
table(youtube_deiden$favorite_count, useNA = 'ifany')
```

```
0 <NA>
231    16
```



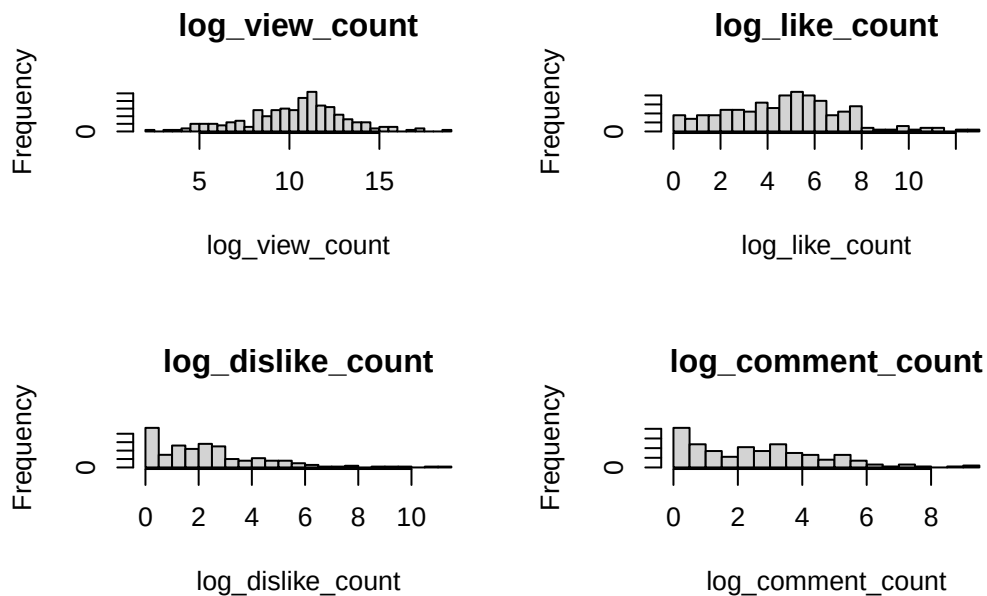
We can see the data are strongly right-skewed except `favorite_count`. Thus, we can apply $\log(1 + x)$. We are not going to use `favorite_count` because most of it are 0.

```
youtube_deiden$log_view_count <- log1p(youtube_deiden$view_count)
youtube_deiden$log_like_count <- log1p(youtube_deiden$like_count)
youtube_deiden$log_dislike_count <- log1p(youtube_deiden$dislike_count)
youtube_deiden$log_comment_count <- log1p(youtube_deiden$comment_count)
# calculate log(1 + x)

outcome_transformed <- c(
  'log_view_count',
  'log_like_count',
  'log_dislike_count',
  'log_comment_count'
)

par(mfrow = c(2, 2))
for(i in outcome_transformed){ # draw the distribution
  hist(
    youtube_deiden[[i]],
    main = i,
    xlab = i,
    breaks = 30
  )
}
```

```
)  
}
```



```
par(mfrow = c(1, 1))
```

The transformed variables are used as the outcomes in the analyses. `favorite_count` is not suitable because it has no variation.

c. Fitting linear regression models

```
predictor <- c(  
  "funny",  
  "show_product_quickly",  
  "patriotic",  
  "celebrity",  
  "danger",  
  "animals",  
  "use_sex"  
)  
# set the characteristics we want to use
```

```

to_int <- function(x){ # change those Boolean expression into integer
  if(is.logical(x)){
    return(as.integer(x))
  }
}

youtube_deiden[predictor] <-
  lapply(
    youtube_deiden[predictor],
    to_int
  )
# Transform logical predictors into 0 and 1

```

```

models <- expand.grid(
  outcome = outcome_transformed,
  predictors = predictor,
  stringsAsFactors = FALSE
)

result_model <- do.call(
  rbind,
  lapply(seq_len(nrow(models)), function(i){
    outcome_i <- models$outcome[i]
    predictors_i <- models$predictors[i]
    # select outcome and predictor for row i

    formula_i <- as.formula(
      paste(
        outcome_i,
        "~",
        predictors_i,
        "+ year"
      )
    ) # construct formula

    fit_i <- lm(
      formula = formula_i,
      data = youtube_deiden
    ) # fit the linear model

    # get the coefficient table from summary

```

```

coef_table <- summary(fit_i)$coefficients

data.frame( # form a data.frame with outcome, predictor, estimate, p_value
  outcome = outcome_i,
  predictor = predictors_i,
  estimate <- coef_table[predictors_i, "Estimate"],
  p_value <- coef_table[predictors_i, "Pr(>|t|)"],
  stringsAsFactors = FALSE
)
}
)
)

names(result_model) <- c(
  "outcome", "predictor", "estimate", "p_value"
)

result_model$significant <- result_model$p_value < 0.05
result_model$direction <- ifelse(result_model$estimate > 0, "positive", "negative")
# add significant and direction
result_model

```

	outcome	predictor	estimate	p_value	significant
1	log_view_count	funny	0.52984356	0.20510610	FALSE
2	log_like_count	funny	0.36019066	0.34005646	FALSE
3	log_dislike_count	funny	0.04148774	0.89580597	FALSE
4	log_comment_count	funny	0.04746590	0.87934573	FALSE
5	log_view_count	show_product_quickly	0.20898031	0.60072225	FALSE
6	log_like_count	show_product_quickly	0.16825538	0.64047730	FALSE
7	log_dislike_count	show_product_quickly	0.24290875	0.42097425	FALSE
8	log_comment_count	show_product_quickly	0.34602814	0.24685774	FALSE
9	log_view_count	patriotic	0.13692497	0.78391866	FALSE
10	log_like_count	patriotic	0.49311233	0.28968290	FALSE
11	log_dislike_count	patriotic	0.66965405	0.08563747	FALSE
12	log_comment_count	patriotic	0.51478460	0.16854269	FALSE
13	log_view_count	celebrity	-0.01898556	0.96288885	FALSE
14	log_like_count	celebrity	0.26297830	0.47787826	FALSE
15	log_dislike_count	celebrity	-0.28325315	0.36162721	FALSE
16	log_comment_count	celebrity	0.22806291	0.45558834	FALSE
17	log_view_count	danger	0.66678076	0.09406479	FALSE
18	log_like_count	danger	0.62526219	0.08031647	FALSE
19	log_dislike_count	danger	0.18514931	0.53794989	FALSE

20	log_comment_count	danger	0.10642262	0.72095533	FALSE
21	log_view_count	animals	-0.17326980	0.64669142	FALSE
22	log_like_count	animals	0.02459257	0.94282914	FALSE
23	log_dislike_count	animals	-0.07378053	0.79742453	FALSE
24	log_comment_count	animals	-0.19176266	0.49906680	FALSE
25	log_view_count	use_sex	-0.29629543	0.49051595	FALSE
26	log_like_count	use_sex	-0.32497453	0.40008928	FALSE
27	log_dislike_count	use_sex	-0.38888066	0.22927926	FALSE
28	log_comment_count	use_sex	-0.34939714	0.27353634	FALSE

direction

1	positive
2	positive
3	positive
4	positive
5	positive
6	positive
7	positive
8	positive
9	positive
10	positive
11	positive
12	positive
13	negative
14	positive
15	negative
16	positive
17	positive
18	positive
19	positive
20	positive
21	negative
22	positive
23	negative
24	negative
25	negative
26	negative
27	negative
28	negative

1. None of the commercial characteristics are statistically significant.
2. All of them are negative.

```

result_view_ct <- subset(result_model, outcome == "log_view_count")
largest_view_ct <- result_view_ct[which.max(result_view_ct$estimate),
                                c("outcome",
                                  "predictor",
                                  "estimate",
                                  "p_value",
                                  "significant",
                                  "direction")]
# get the row with outcome is log_view_count and largest estimate value
largest_view_ct

```

```

      outcome predictor  estimate  p_value significant direction
17 log_view_count   danger 0.6667808 0.09406479      FALSE  positive

```

3. Commercial characteristic danger has the largest positive estimated association with view count

d. Estimating regression coefficients using matrix algebra

```

result_view <- subset(
  result_model,
  outcome == "log_view_count"
)
# get regression results for log_view_count only
result_view

```

```

      outcome      predictor  estimate  p_value significant
1 log_view_count      funny 0.52984356 0.20510610      FALSE
5 log_view_count show_product_quickly 0.20898031 0.60072225      FALSE
9 log_view_count    patriotic 0.13692497 0.78391866      FALSE
13 log_view_count    celebrity -0.01898556 0.96288885      FALSE
17 log_view_count     danger 0.66678076 0.09406479      FALSE
21 log_view_count    animals -0.17326980 0.64669142      FALSE
25 log_view_count    use_sex -0.29629543 0.49051595      FALSE
direction
1 positive
5 positive
9 positive
13 negative

```

```
17 positive
21 negative
25 negative
```

```
selected_pred <- result_view$predictor[which.min(result_view$p_value)]
# get the commercial characteristic with the smallest p-value
selected_pred
```

```
[1] "danger"
```

```
variables <- c("log_view_count", selected_pred, "year")
# choose the variables we need
data_ols <- youtube_deiden[
  complete.cases(youtube_deiden[variables]),
  variables,
  drop = FALSE
] # keep only these variables
head(data_ols)
```

	log_view_count	danger	year
1	12.066408	0	2018
2	10.773797	1	2020
3	11.865770	1	2006
4	5.293305	0	2018
5	9.528212	1	2003
6	10.070569	1	2020

```
X <- cbind(
  intercept = 1,
  predictor = data_ols[[selected_pred]],
  year = data_ols$year
)
# construct X matrix in OLS

y <- data_ols$log_view_count

dim(X)
```

```
[1] 231 3
```

```
head(y)
```

```
[1] 12.066408 10.773797 11.865770 5.293305 9.528212 10.070569
```

```
length(y)
```

```
[1] 231
```

```
# construct y in OLS
```

```
# build beta_hat
beta_hat <- solve(t(X) %*% X) %*% (t(X) %*% y)
rownames(beta_hat) <- c(
  "(Intercept)",
  selected_pred,
  "year"
)

beta_hat
```

```
          [,1]
(Intercept) -31.39149243
danger       0.66678076
year        0.02065641
```

```
# print beta_hat
```

```
formula_OLS <- as.formula( # construct the formula
  paste(
    "log_view_count ~",
    selected_pred,
    "+ year"
  )
)
fit_OLS <- lm(formula = formula_OLS, data = data_ols)
# fit the formula with function lm()

coef(fit_OLS)
```

(Intercept)	danger	year
-31.39149243	0.66678076	0.02065641

```
# get beta_hat from lm()
```

```
all.equal(as.numeric(beta_hat), as.numeric(coef(fit_OLS)))
```

```
[1] TRUE
```

```
# test if our beta_hat is equal to the one from lm()
```

The result of testing if $\hat{\beta}$ and coefficient from `lm()` is equal is TRUE